How-To Library

Informatica

Performance Tuning and Best Practices for Snowflake Data Cloud Connector

© Copyright Informatica LLC 2019, 2026. Informatica, Informatica Cloud, and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

When you use Snowflake Connector to read data from or write data to Snowflake, multiple factors such as hardware, database, Amazon EC2, Secure Agent, and Informatica mapping parameters impact the connector performance. You can optimize the performance by tuning these parameters appropriately. This article describes general reference guidelines to help you tune the performance of Snowflake Connector in Cloud Data Integration.

Supported Versions

- Informatica® Cloud Data Integration

Table of Contents

Overview. 2
Performance Tuning Areas. 3
Amazon EC2 Best Practices. 3
Tuning the Secure Agent. 3
JVM Heap Size. 3
Client Memory Limit. 4
Tuning the Snowflake Database. 4
Compress Files. 5
Dedicate Separate Warehouses for Snowflake Load and Query Operations. 5
Tuning the Snowflake Mapping. 5
Partitions. 5
Number of staging files. 7
CSV File Size. 8
Local flat file staging. 8

Overview

Performance tuning is an iterative process in which you analyze the performance, use guidelines to estimate and define parameters that impact the performance, and monitor and adjust the results as required.

This document describes the key hardware, database, Amazon EC2, Secure Agent, and Informatica mapping parameters that you can tune to optimize the performance of Snowflake Data Cloud Connector. This document also includes case studies that involve the tuning specifications required when you configure a mapping to write data to Snowflake from a flat file using Amazon EC2 and the Secure Agent. The graphs constructed from the case studies illustrate the impact that the tuning has on Snowflake Data Cloud Connector performance.

Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of Snowflake Data Cloud Connector might vary based on individual environments and other parameters even when you use the same data.

Performance Tuning Areas

You can optimize the performance of a read or write operation when you use Snowflake Data Cloud Connector by tuning the following parameters:

- Amazon EC2
- Secure Agent
- Snowflake database
- Snowflake mapping

Amazon EC2 Best Practices

To optimize the performance, consider the following best practices for Amazon EC2:

- Ensure that the Secure Agent hosted on the EC2 machine is in the same region as Snowflake.
- Choose Snowflake in the region where the Amazon EC2 is located.
- Choose the right EC2 instance type based on your requirements.

Tuning the Secure Agent

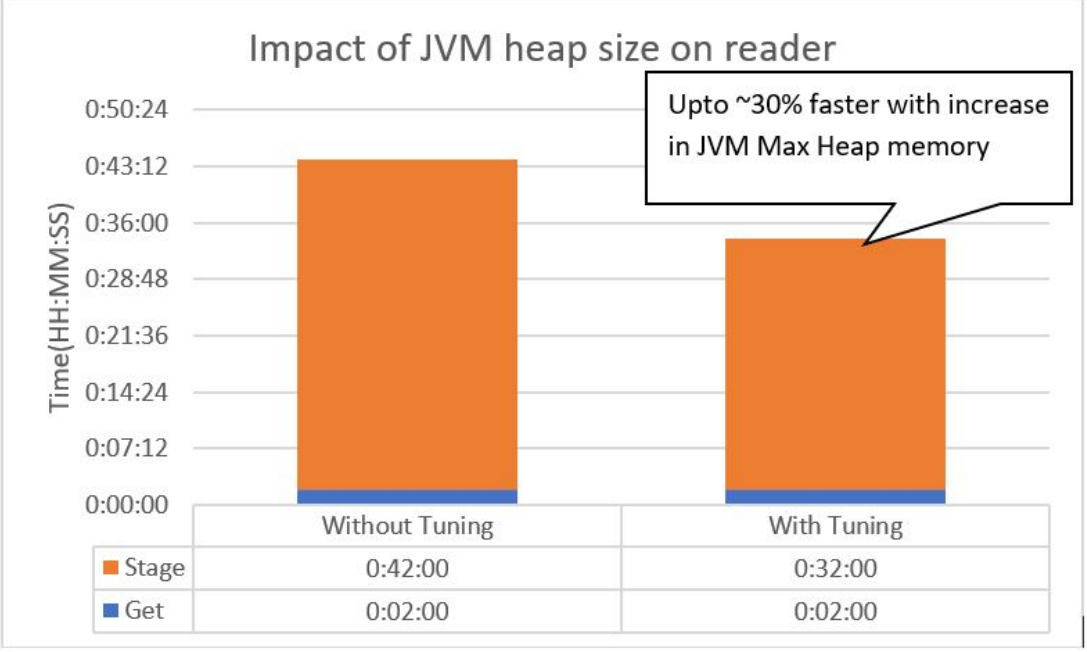
You can tune the JVM heap size and the client memory limit for the Secure Agent to optimize the performance of a Snowflake read or write operation.

JVM Heap Size

You can specify the amount of memory that the Secure Agent uses for each pmdtm process. Default is 64 MB.

If you use the default JVM heap size, the mapping fails with an out-of-memory error. Increasing the JVM heap size improves the performance of the read and write operations.

The following graph shows the results and the recommended values for setting the JVM maximum heap size:



For Snowflake mappings and mapping tasks that read data from Snowflake, specify a JVM memory of -Xmx256m.

If you already configured a heap size value that is higher than 256 MB in the JVM options, do not change it. -Xmx256m

Perform the following steps to configure the JVM memory:

1. In Administrator, select the Secure Agent listed on the **Runtime Environments** tab.
2. Click **Edit**.
3. In the **System Configuration Details** section, select **Data Integration Service** as the service and **DTM** as the type.
4. Edit the **JVMOption1** property, and enter *-Xmx256m*.
5. Click **Save**.

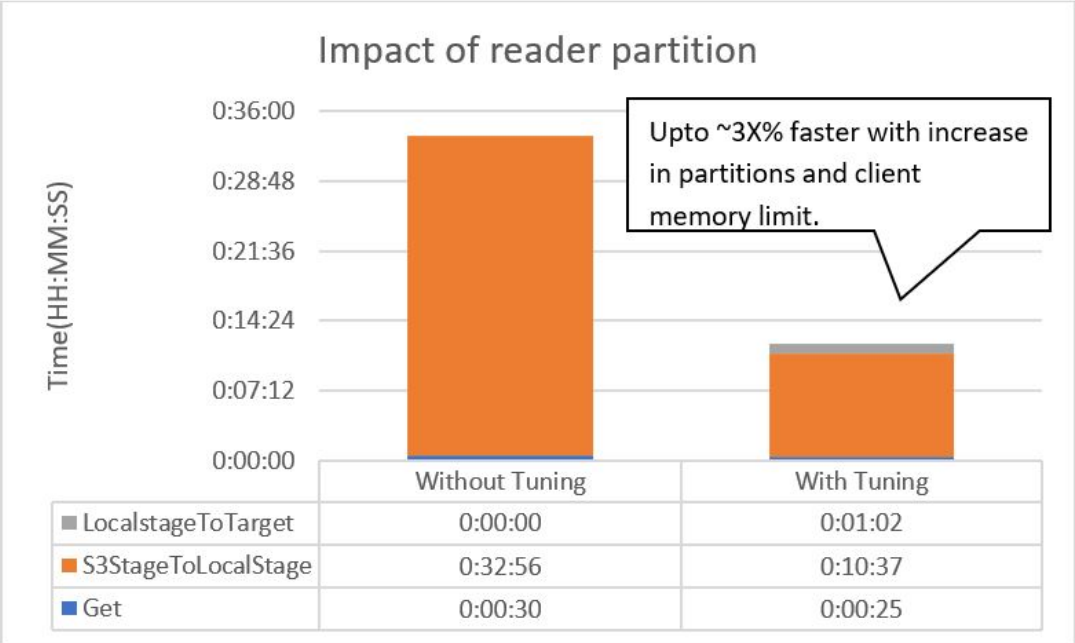
Client Memory Limit

You can specify the maximum amount of memory in MB that the JDBC driver must use for each thread or partition.

To ensure that sufficient memory is available for each partition, you must set the CLIENT_MEMORY_LIMIT significantly higher than the chunk size of the results set. Set the CLIENT_MEMORY_LIMIT property in the **Additional JDBC URL Parameters** field in the Snowflake connection. For example, `CLIENT_MEMORY_LIMIT=8192`

The CLIENT_MEMORY_LIMIT property simplifies the JVM memory management and sets a global maximum memory usage limit for all queries. When you increase the client memory limit for the Secure Agent in conjunction with the number of partitions, the reader performance increases. If you do not set the CLIENT_MEMORY_LIMIT limit, the reader partitioning does not scale the throughput linearly.

The following graph illustrates the impact that the client memory limit has on the reader performance:



You can view the results and the recommended values for setting the client memory limit.

Tuning the Snowflake Database

You can tune the Snowflake components to optimize the read and write performance.

Compress Files

The number of load operations that run in parallel must not exceed the number of data files that you want to load. To optimize the number of parallel operations for a load, you can compress data files to a size between 10 MB and 100MB. The number of data files that are processed in parallel is determined by the number and capacity of servers in a warehouse.

Dedicate Separate Warehouses for Snowflake Load and Query Operations

When you configure a mapping to load large data sets, the query performance can get impacted. To optimize performance when you configure load and query operations, you must dedicate separate warehouses. The number and capacity of servers in a warehouse determine the number of data files that can be processed in parallel.

You must follow the file sizing guidelines so that the data load uses minimal resources. For more information, see topic "Preparing Your Data Files" in the following website:
https://docs.snowflake.net/manuals/user-guide/data-load-considerations-prepare.html

You can split larger data files to allow the load to scale linearly. A smaller warehouse is generally sufficient unless you want to concurrently load a large number of data ranging from a few hundreds to thousands of files. A larger warehouse such as X-Large or 2X-Large consumes more credits and might not optimize the performance.

To understand the impact of dedicating warehouses to process queries, refer to the Snowflake guidelines and make appropriate changes depending upon the complexity of data and the amount of data that you want to process.

Tuning the Snowflake Mapping

You can configure partitions and local staging files to optimize Snowflake mapping performance.

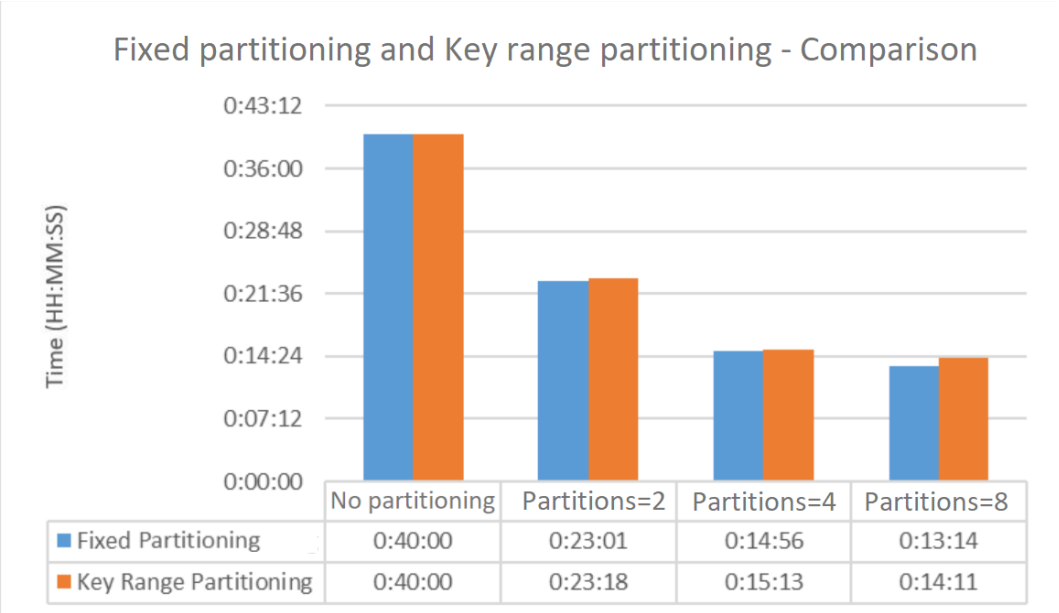
Partitions

You can configure partitioning to optimize mapping performance at run time when you read from or write to Snowflake.

For more information about configuring source and target partitioning in a mapping, see the *Snowflake Data Cloud Connector* help.

Source partitioning

The following image shows the execution time comparison for mappings configured with fixed partitioning and key range partitioning:



Use key range partitioning when your dataset has a field with values that can be divided into even ranges, such as a date or numeric field. Without a well-distributed partition key, key range partitioning can result in unevenly distributed partitions that negatively affect performance.

Use fixed partitioning in the following situations:

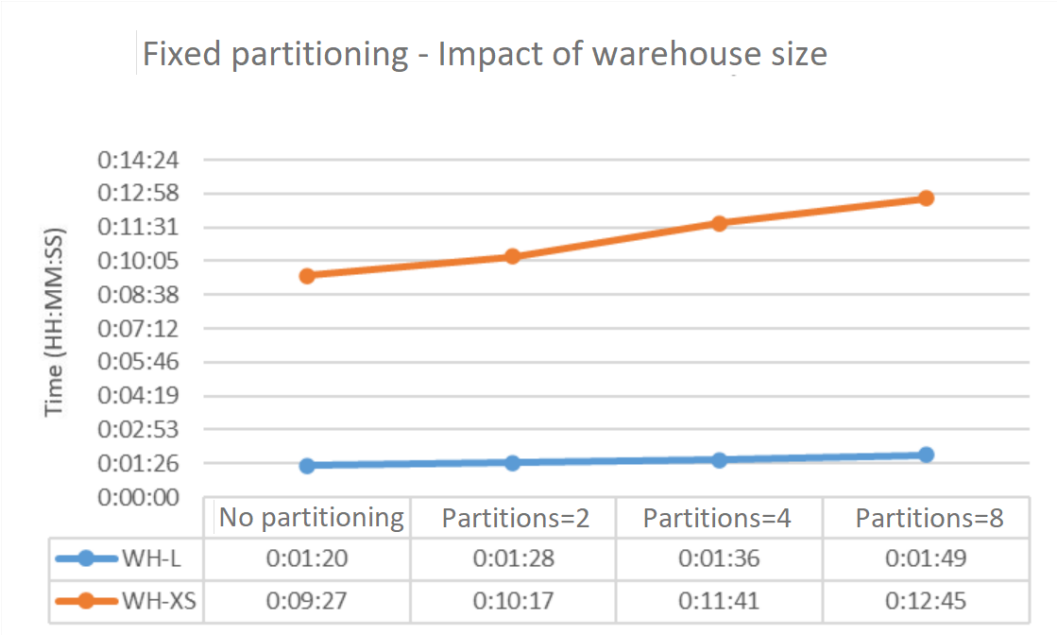
- Your dataset doesn't have a field that is suitable for key range partitioning.
- Your partition key isn't a date or numeric data type.
- You want to create partitions based on a combination of fields in your dataset.

When you configure fixed partitioning in the Source transformation on a Large warehouse, increase the number of partitions to eight to improve performance.

To improve performance on an X-Small-sized warehouse, reduce the number of partitions. If you increase the number of partitions, the mapping increases the number of concurrent COPY command executions on the warehouse, which can degrade performance.

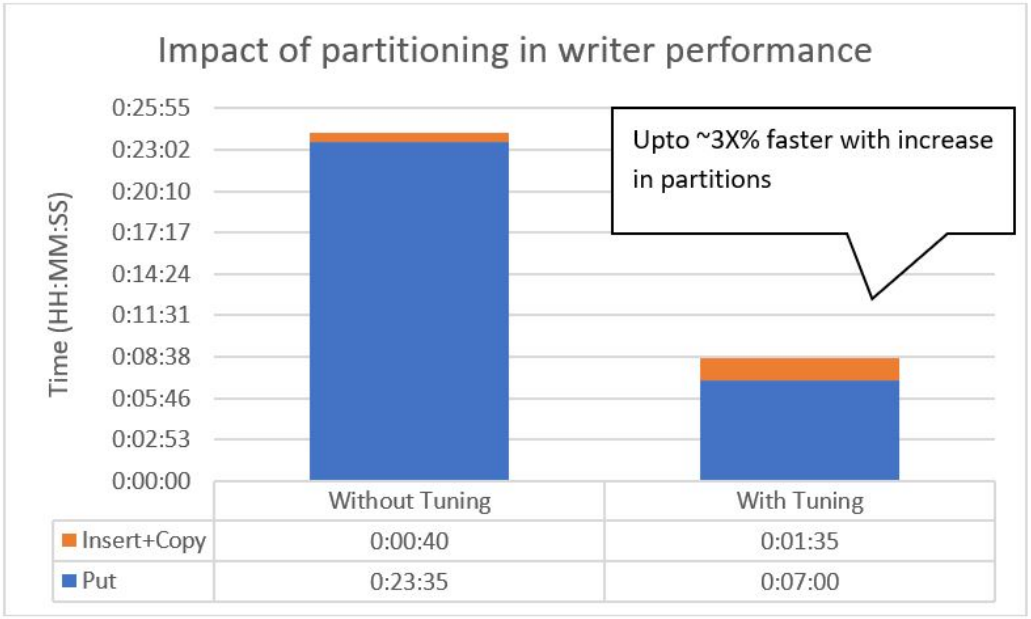
For more information about the warehouse size, see Snowflake warehouse size.

The following image shows the execution time comparison for mappings configured with fixed partitioning, based on the warehouse size:



Target partitioning

The following image shows the impact of partitioning on the writer performance:



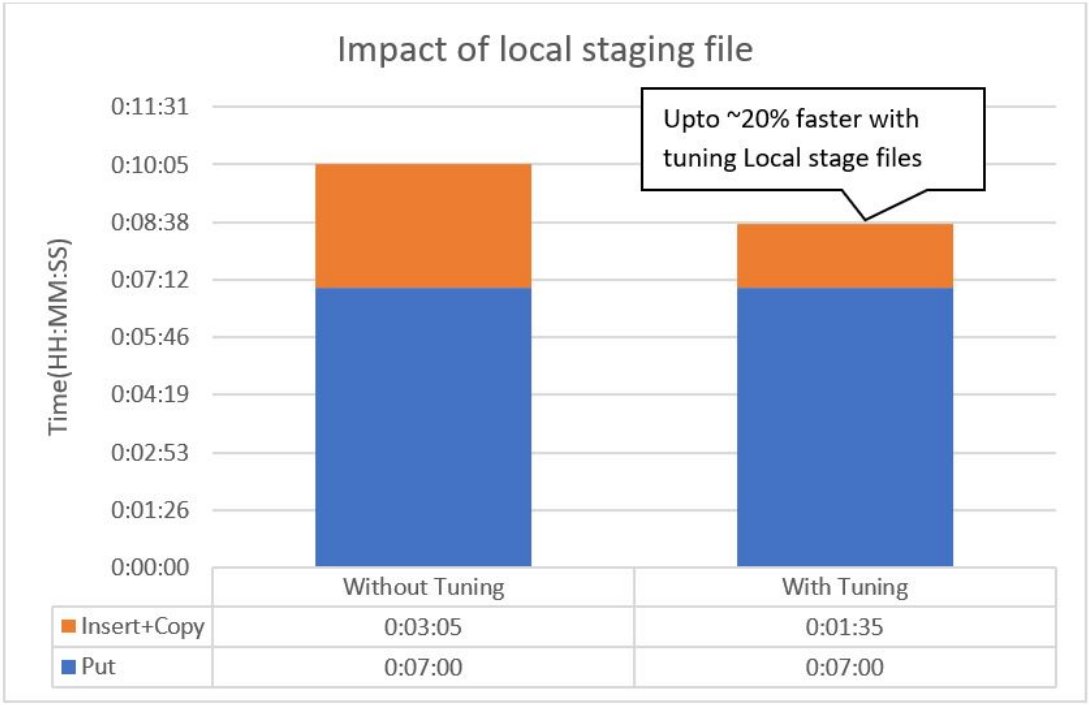
Number of staging files

You can optimize a Snowflake mapping performance by specifying the number of local staging files in the Snowflake Target transformation. The agent writes data to the target after the specified number of local staging files are created.

If the copy commands such as create temp, insert, and remove commands are faster than the PUT command, the performance improves when there are lesser number of local stage files.

For better performance, increase the number of partition and the batch row size value, and keep the default local stage files. For example, if the number of partitions = 1 and the number of local stage files =1, this combination results in better performance, which again is subject to the warehouse type and network latency.

The following graph illustrates the optimization of writer performance when you configure staging files in the Snowflake Target transformation:



CSV File Size

When you create a mapping to write to Snowflake, you can specify the size of the local staging .csv file in bytes. Specify the local staging file size property, csvFileSize, in the **Additional Write Runtime Parameters** field in the advanced Snowflake target properties. The default file size is 50 MB.

If the intended file size is 50 MB, calculate the csvFileSize value in bytes, for example 50*1024*1024 and then specify 52428800 as the csvFileSize. It is recommended that you configure the right combination of the number of local staging files and csvFileSize while writing data to Snowflake for better performance.

Local flat file staging

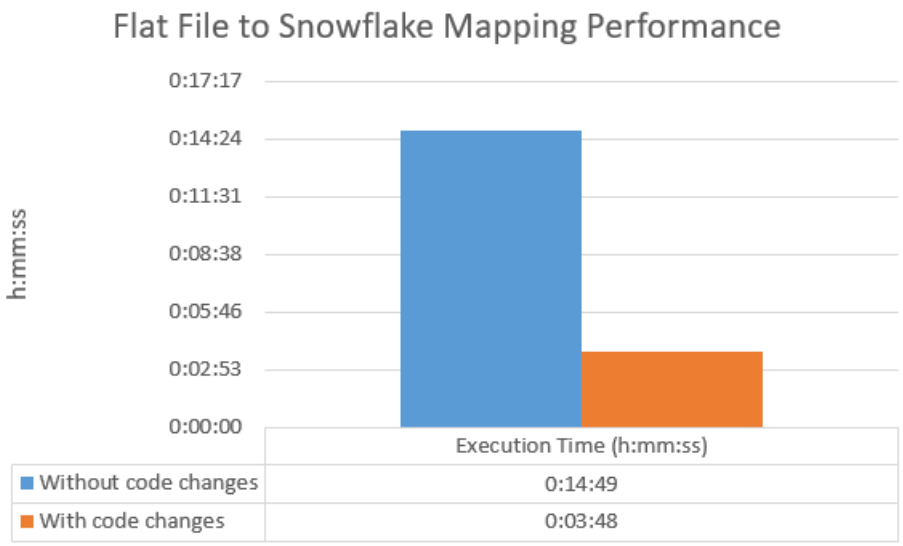
Informatica has redesigned the way Data Integration stages the data when you read from or write to a Snowflake target. This has significantly improved the performance in the local staging execution time.

You can optimize the mapping performance by setting the staging property in the Secure Agent to create a flat file in a temporary folder to stage the data locally before reading from or writing to Snowflake.

Writing to a Snowflake target

When you set the staging property *INFA_DTM_STAGING_ENABLED_CONNECTORS* for the Tomcat to the *plugin ID* of Snowflake Data Cloud Connector in the Secure Agent properties, Data Integration, by default, creates a flat file locally to stage the data and then loads the data from the staging file to the Snowflake target. For more information on how to set the property, see the help for Snowflake Data Cloud Connector.

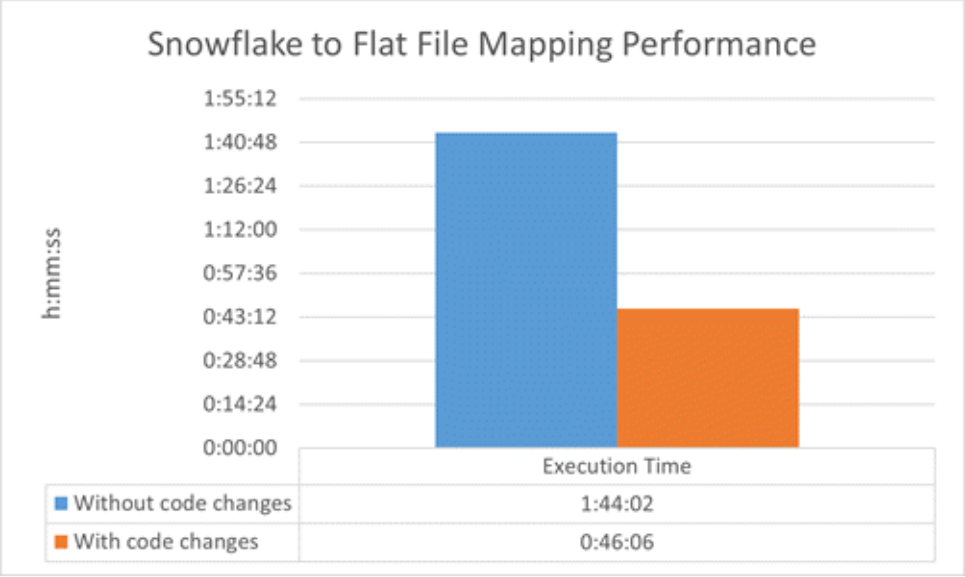
The following image shows the performance comparison of the execution time before and after setting the staging property for a mapping that reads from a flat file and writes to Snowflake:



Reading from a Snowflake source

When you set the staging property *INFA_DTM_RDR_STAGING_ENABLED_CONNECTORS* for the Tomcat to the plugin ID of the Snowflake Data Cloud Connector in the Secure Agent properties, Data Integration, by default, creates a flat file locally to stage the data from the Snowflake source and then reads the data from the staging file. For more information on how to set the property, see the help for Snowflake Data Cloud Connector.

The following graph shows the performance comparison of the execution time before and after setting the staging property for a mapping that reads from a Snowflake source and writes to a flat file:



CPU and memory requirements

Snowflake Data Cloud Connector uses asynchronous processing to optimize processing performance. It concurrently writes the staging files to the temporary staging directory and simultaneously uploads the staged files to the Snowflake internal stage. Because CPU is consumed to both stage the files and also to upload files to the internal stage, the CPU usage is higher. Informatica recommends that you add six processor cores to the agent machine to increase the processing power of the mapping.

The following table shows the resource sizing in the old and new environment for a mapping that reads from a flat file source and writes to a Snowflake target:

Staging	CPU requirement (cores)	Memory requirement (GB)
Without code changes	3	2 GB (JVMHeap - 1 GB and DTM Buffer Size 1 GB)
With code changes	6	2 GB (JVMHeap - 1 GB and DTM Buffer Size 1 GB)

The following table compares the total end-to end execution time with the available CPU cores on the agent machine for a mapping that reads from a flat file source and writes to a Snowflake target:

Available CPU Cores	CPU Cores Used	CPU Utilization (%)	Execution Time
8	6	75	0:03:48
4	4	100	0:05:55

When you use six cores, the performance is optimized and the local staging execution time is far lesser. Though Informatica recommends that you include six cores for a Snowflake mapping execution, it is not mandatory. However, when you include lesser cores, the mapping performance is impacted but it does not result in execution failures.

Authors

Mridul Mathew

Dimple Rai